Python: the grammar behind what you've been doing

Syntax and constructs, from values to functions

Two ways to learn a language

Immersion

What we did in Colab: run it, change it, see what happens.

```
words = sample.split()
words[-3:]
→ ['knew', 'by', 'noon.']
```

The grammar book

Today: name the parts, see the patterns.

```
thing.method(argument)
list[start:stop]
```

Both help. Each makes sense of the other.

Every value has a type

`str`	text	`"Bennet"`
`int`	whole number	`39`
`float`	decimal number	`4.2`
`list`	ordered collection	`['Mr.', 'Bennet']`
`bool`	true or false	`True`

```
type("Bennet")
→ str

"10" + "5"
→ '105'

10 + 5
→ 15
```

= gives a name; == asks a question

= assignment

Make this name refer to this value.

== comparison

Are these equal? The answer is True or False.

```
size = len(sample)

size == 196
→ True

name = "Bennet"
name == "bennet"
→ False
```

Three ways to call a tool

FUNCTION

`function(thing)`	`len(sample)`	*do len to sample*

METHOD

`thing.method()`	`sample.split()`	*take sample and split it*

MODULE FUNCTION

`module.function()`	`statistics.mean(x)`	*mean, from the statistics module*

`sample.split()` vs `re.split(pattern, sample)`: same job, different tool

What goes in, what comes back

```
# one argument
len(sample)
→ 196

# two arguments
sample.replace("Bennet", "Darcy")
→ 'Mr. Darcy looked up ...'

# the original is unchanged
sample
→ 'Mr. Bennet looked up ...'
```

Arguments

go inside the brackets, separated by commas.

Return values

are what comes back. Keep one by naming it.

`print(x)` shows x, but gives back nothing.

Strings: text and its methods

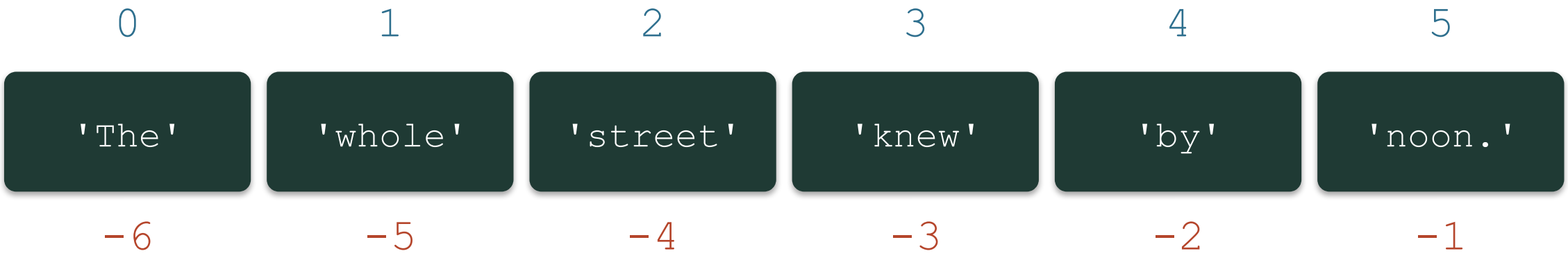
Quotes must match

```
'Mr. Bennet'
"my dear?"
'"Is it?" he said.'
```

Text with double quotes goes inside single quotes.

`.lower()`	lower case
`.upper()`	capitals
`.count("x")`	how many x
`.replace("a", "b")`	swap a for b
`.split()`	list of words
`.strip(".,")`	trim the ends

Lists: counting from 0, and from the end




```
ending[0]      → 'The'
ending[-1]     → 'noon.'
ending[1:3]    → ['whole', 'street']
ending[-3:]    → ['knew', 'by', 'noon.']
```

`[start:stop]`

stops *before* stop

Leave a number out

start or end of the list

List comprehensions

```
[len(w) for w in words if len(w) > 5]
```

what to make

the length of each word

from where

each item in turn, called w

which ones

keep only items that pass (optional)

```
[w.upper() for w in words]                    # change every item
[w for w in words if w.startswith("B")]       # keep some items
→ ['Bennet', 'Bennet']
```

for loops: do something for each item

```
for w in words[0:3]:
    print(w, len(w))
→ Mr. 3
→ Bennet 6
→ looked 6
```

The colon

ends the first line

The indent

four spaces: what is repeated

Loop or comprehension?

Do something: loop. Make a list: comprehension.

Asking questions: True or False

`==`	equal
`!=`	not equal
`<  >`	less, greater
`<= >=`	or equal to
`in`	inside?
`not in`	absent?

```
"Bennet" in words
→ True

if len(w) > 5:
    print("long:", w)
else:
    print("short:", w)
```

Writing your own functions

```
def clean(w):
    return w.strip('.,;:!?"').lower()

clean('"Bennet,')      → 'bennet'
```

`def`	starts a definition	`(w)`	the input: whatever is passed in
`:`	then an indented body	`return`	what comes back

Modules, packages, libraries

module

one file of code: `re`

package

a collection of modules: `nltk`

library

either, informally: `pandas`

```
import statistics
statistics.mean([3, 6, 6, 2, 4])
→ 4.2

import nltk
nltk.download('punkt_tab')

# in Colab, if a package is missing:
!pip install something
```

Error messages: read the last line

Error	Means	Example
`NameError`	unknown name	`Print(x)`
`AttributeError`	no such method	`sample.Upper()`
`IndexError`	past the end of a list	`words[100]`
`TypeError`	wrong type	`"Page " + 5`
`SyntaxError`	not valid Python	`print("hi)`

The worst error has no message: *wrong answers, quietly.*

Habits for working with AI-written code

1. **Test it on something you can check by hand**
   a few sentences where you know the answer
2. **Ask what it assumed**
   what is a word, a sentence, the text?
3. **Read its explanation against the code**
   explanations can be confidently wrong
4. **Ask twice, ask another AI**
   same prompt, same answer?
5. **Report your decisions**
   in your methods, not hidden in code

Your turn: ask an AI

“Write Python code for Google Colab that downloads Pride and Prejudice from Project Gutenberg and lists the 20 most common words.”

Does it run? What did it read? What counts as a word? Did it remove anything? How many times does it count “pride”?